Operating Systems Note

Contents

1 Introduction **1**
- 1.1 Execution Model . . . 1
- 1.2 Main Memory vs. Hard Disk . . . 1

2 Interrupts **2**
- 2.1 Basics . . . 2
- 2.2 What Happens on an Interrupt . . . 2
- 2.3 Why Interrupts Matter . . . 2

3 x86 Privilege Rings **3**

4 Number Representation (Endianness) **3**
- 4.1 Example . . . 3

5 Function Calls and the Stack **3**
- 5.1 Saving Registers . . . 3
- 5.2 Example: One-Register Machine . . . 4

6 Booting **4**
- 6.1 Power-On . . . 4
- 6.2 Boot Sequence . . . 5
- 6.3 Master Boot Record (MBR) . . . 5

A Glossary **6**

1 Introduction

1.1 Execution Model

- **Assumption:** a machine with a single-core CPU, one program counter (`PC`) and one instruction register (`IR`).
- The CPU repeatedly performs the *fetch–execute* cycle: `PC` holds the address of the next instruction, that instruction is fetched into `IR`, and then it is executed.
- When an application is running, the CPU executes the application's instructions directly on the hardware, not on top of the OS. This is called running on **bare metal**.
- The OS is *not* executing while an application runs. It only gets the CPU back through an interrupt or a system call (see Section 2).
- So, for native (non-interpreted) code, whatever is currently running is running on bare metal.

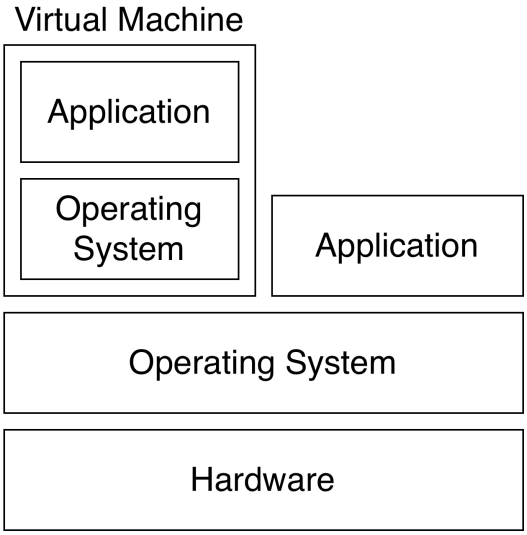


Figure 1: The usual layered depiction

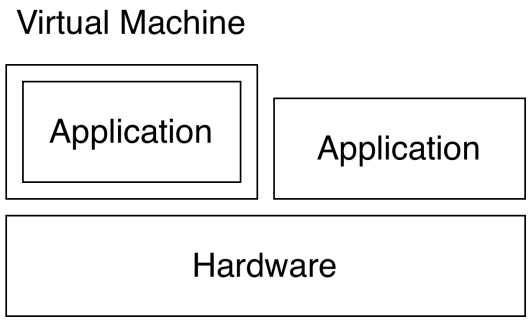


Figure 2: What is actually happening: the running code executes directly on the hardware

Interpreters and virtual machines

Code run by an interpreter (e.g. Python, Java bytecode in a JVM) does not itself run on bare metal. The interpreter does, and it executes the code on its behalf. With hardware-assisted virtualization, a guest OS's instructions typically also run directly on the CPU, with a hypervisor stepping in only when needed.

1.2 Main Memory vs. Hard Disk

- Main memory is **byte addressable**.
- A hard disk is accessed in blocks called **sectors**. Positioning on a disk (seek and rotation) is slow compared to memory, so transferring one byte at a time would waste CPU time. Data is therefore moved in fixed-size chunks.
- Both are typically drawn as an array of slots for ease of understanding.

	Main memory (RAM)	**Hard disk**
Smallest access unit	1 byte	1 sector (commonly 512 B)
Relative speed	Fast	Slow
Volatile?	Yes	No

2 Interrupts

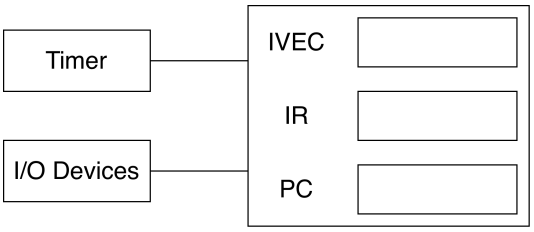


Figure 3: Devices connect to the CPU through interrupt lines

2.1 Basics

- Interrupts are signals generated by I/O devices (keyboard, mouse, ...) or by the timer. These devices are connected to the CPU through interrupt lines.
- The `IVEC` (interrupt vector) register holds the memory address of the first instruction of the interrupt handler. The handler is part of the OS.

2.2 What Happens on an Interrupt

1. An application is running; the CPU is fetching its instructions.
2. An interrupt signal arrives on an interrupt line.
3. The value in `IVEC` is copied into `PC`.
4. The next fetch therefore reads the handler's first instruction (from the OS's memory) into `IR`.
5. The handler executes, so the OS is now running.

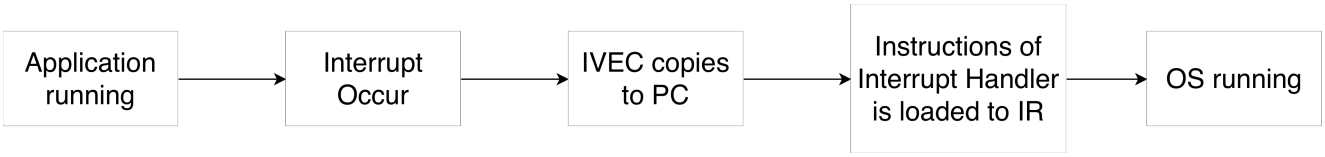


Figure 4: Control transfer from application to OS on an interrupt

2.3 Why Interrupts Matter

- Without interrupts, an application could run forever and hog the CPU.
- Keyboard and mouse interrupts happen only when the user acts, so they cannot be relied on. The **timer** therefore raises an interrupt at regular intervals, guaranteeing that the OS regains control periodically.

Common misconception

The interrupt changes `PC`, not `IR`. `IR` merely holds the instruction currently being executed; it receives the handler's first instruction on the next fetch.

Real hardware

A single `IVEC` register is a simplification. On x86 there are many interrupt sources, each with its own handler, so the CPU uses an *interrupt vector table* (the IDT in protected mode), which holds one handler address per interrupt number. The CPU also saves the old `PC` and flags and switches to kernel mode before running the handler.

3 x86 Privilege Rings

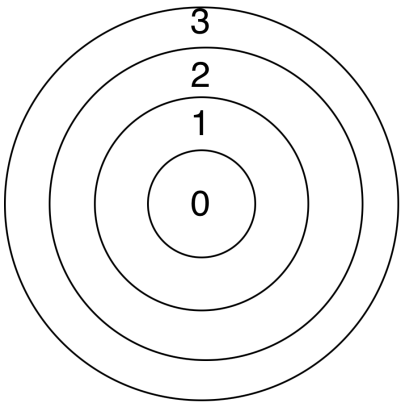


Figure 5: x86 protection rings

- The rings are privilege levels: **0 is the most privileged, 3 the least**.
- The CPU uses them to restrict which memory can be accessed and which instructions can be executed, so that buggy or malicious software cannot crash the system.
- **Ring 0 – Kernel mode:** the OS kernel runs here with unrestricted access to memory and hardware.
- **Rings 1 & 2:** intended for device drivers and OS services needing intermediate privilege.
- **Ring 3 – User mode:** applications and user-space processes run here with restricted hardware access and isolated virtual memory.

In practice

Mainstream operating systems (Windows, Linux) use only **ring 0 and ring 3**. Rings 1 and 2 exist in the architecture but are essentially unused.

4 Number Representation (Endianness)

- A number that spans multiple bytes has a **Most Significant Byte (MSB)** and a **Least Significant Byte (LSB)**.
- *Endianness* defines the order in which these bytes are stored in memory.
- **Big endian:** the MSB is stored at the *lowest* address.
- **Little endian:** the LSB is stored at the *lowest* address.
- x86 uses **little endian**.

4.1 Example

Consider the 4-byte number `0x12345678`, with MSB = `12` and LSB = `78`.

Address	`+0`	`+1`	`+2`	`+3`
Big endian	`12`	`34`	`56`	`78`
Little endian	`78`	`56`	`34`	`12`

Why it matters later

Endianness affects multi-byte fields on disk too. For example, the MBR signature (Section 6.3) is the byte sequence `55 AA`, which reads as the 16-bit value `0xAA55` on a little-endian machine.

5 Function Calls and the Stack

5.1 Saving Registers

- When a function is going to use a register that already holds a value the caller needs, the compiler emits instructions that `push` the register onto the stack at the start of the function and `pop` it back at the end.
- This preserves the caller's values.
- The exact rules (whether caller or callee saves which registers) are defined by the **calling convention**.

Common misconception

Declaring a variable does not by itself push a register. Local variables are either kept in registers or allocated in the function's stack frame (by adjusting the stack pointer). Pushes happen to *save registers that the function will overwrite.*

5.2 Example: One-Register Machine

```c
#include <stdio.h>

void func() {
        int x = 0;
        x++;
        printf("%d\n", x);
}

int main() {
        int b = 10;
        func();
        printf("%d\n", b);
        return 0;
}
```

The output is `1` followed by `10`.

Assume the machine has only one register, `R`, for holding variable values.

1. `main` runs first and stores `b = 10` in `R`.
2. The call instruction pushes the **return address** onto the stack, then sets `PC` to the address of the first instruction of `func`.
3. At the start of `func`, before `R` is reused for `x`, the old value of `R` (10) is pushed onto the stack.
4. `func` sets `R` to 0, increments it to 1 and prints it.
5. Before returning, `func` pops the saved value back into `R`, so `R` holds 10 again.
6. The return instruction pops the return address into `PC`, and `main` resumes where it left off and prints 10.

Stack (top first)	State
`saved R = 10`	after the push inside `func`
`return address`	pushed by the call

Values are popped in the reverse order they were pushed (LIFO): the saved register first, then the return address.

6 Booting

6.1 Power-On

- RAM contents are undefined at power-on. Most registers also hold arbitrary values, but the hardware resets a few (including `PC`) to defined values.
- `PC` is set to a fixed **reset vector** in ROM. ROM is non-volatile, so unlike RAM its contents survive power loss.
- On classic x86 the reset vector is 16 bytes below the top of the address space. These 16 bytes hold a jump to the entry code of the **BIOS** firmware. This indirection allows different machines to have different ROM sizes and layouts.
- The BIOS runs in real mode with unrestricted hardware access (the privilege rings do not apply yet).

6.2 Boot Sequence

1. **POST** (Power-On Self Test): the BIOS checks that the hardware is present and working.
2. The BIOS reads the **first sector** of the boot disk into RAM. This is the MBR (Section 6.3).
3. The BIOS jumps to the start of that code in RAM (the *boot call*).
4. The MBR code loads a **bootloader** (directly, or via a second stage or the active partition's boot sector).
5. The bootloader loads the **OS kernel** into memory and jumps to it.
6. The kernel sets up its own interrupt handlers (writes `IVEC` / the interrupt vector table) and finishes initialization.

UEFI

Modern machines use UEFI firmware instead of the legacy BIOS and typically a GPT partition scheme instead of the MBR. The overall idea (firmware → bootloader → kernel) is the same.

6.3 Master Boot Record (MBR)

Consider a disk with 512-byte sectors. The MBR is the first sector.

Offset	Size	Content
0	446 bytes	Boot code
446	64 bytes	Partition table (4 entries × 16 bytes)
510	2 bytes	Boot signature: `55` at 510, `AA` at 511 (written 0x55AA)

- The signature marks the sector as bootable. Read as a little-endian 16-bit word it is `0xAA55`.
- The table holds up to **4 primary partitions**.

Field	Size	Meaning
Status	1 B	`0x80` = active (bootable), `0x00` = inactive
Start CHS	3 B	Start address in cylinder/head/sector form
Type	1 B	Partition type code
End CHS	3 B	End address in CHS form
Start LBA	4 B	First sector, as a logical block address
Sector count	4 B	Size of the partition in sectors

Partition entry (16 bytes).

- There is no stored partition number; it is implied by the entry's position in the table.
- The type is a *code*, not a filesystem name. For example `0x07` is NTFS/exFAT and `0x83` is Linux (used for ext4). A code can map to several filesystems.

Magic numbers

A **magic number** is a fixed byte pattern at the start of a file that identifies its format. On Linux, the kernel uses it to decide how to run an executable, and the `file` command uses it (plus other heuristics) to identify files. The file extension does not matter.

- ELF executables: `0x7F` followed by `ELF`
- Java class files: `0xCAFEBABE`
- Scripts: `#!` (the magic) followed by an interpreter path, e.g. `#!/bin/bash`. The path can be any interpreter.

A Glossary

Term	Meaning
`PC`	Program counter: address of the next instruction
`IR`	Instruction register: the instruction being executed
`IVEC`	Interrupt vector register: address of the interrupt handler
BIOS	Firmware in ROM that starts the boot process
POST	Power-On Self Test, run by the BIOS
MBR	Master Boot Record: first sector of a disk
MSB / LSB	Most / least significant byte
LIFO	Last in, first out (stack order)